

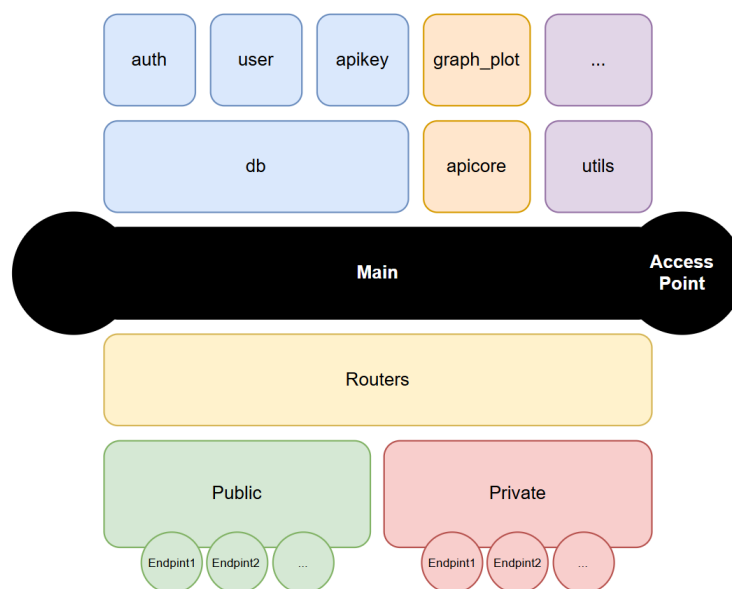
Mestrado em Engenharia e Ciência de Dados

Visualização de Dados e Informação

Ficha tutorial 8 – VDI como um serviço

1. Instale os seguintes pacotes, via *pip*:
 - a. `pip install zipper`
 - b. `pip install uvicorn`
 - c. `pip install fastapi`
 - d. `pip install python-multipart`

Iremos, de seguida, montar uma solução de serviços baseada em FastAPI, replicando a seguinte arquitetura:



2. Crie um projeto com o nome **`__VDI_as_a_service`** com os ficheiros ***main.py*** e ***logging.conf***.
 - a. Coloque o seguinte conteúdo no *main.py*.

```
import uvicorn
from fastapi import FastAPI
import logging
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(debug=True)

app.add_middleware(
    CORSMiddleware,
```

```

allow_origins=["*"], # Allows all origins
allow_credentials=True,
allow_methods=["*"], # Allows all methods
allow_headers=["*"], # Allows all headers
)

@app.get("/")
async def root():
    return {"message": "Hello! I'm gonna rock your visualizations!"}

if __name__ == "__main__":

    logging.config.fileConfig('logging.conf', disable_existing_loggers=False)
    logger = logging.getLogger("sampleLogger")
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8000,
        log_level=logging.INFO,
        reload=True
    )

```

b. Coloque o seguinte conteúdo no *logging.conf*.

```

[loggers]
keys=root,sampleLogger

[handlers]
keys=consoleHandler

[formatters]
keys=coloredFormatter

[logger_root]
level=DEBUG
handlers=consoleHandler

[logger_sampleLogger]
level=DEBUG
handlers=consoleHandler
qualname=sampleLogger
propagate=0

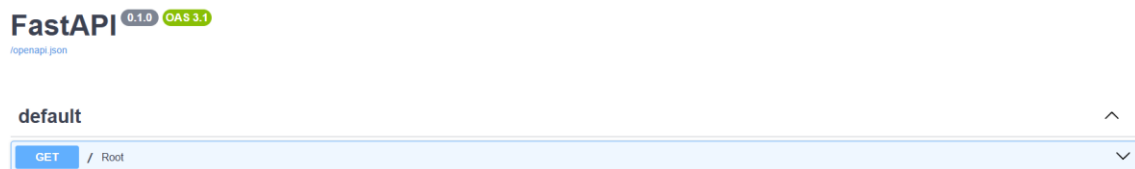
[handler_consoleHandler]
class=StreamHandler
level=DEBUG
formatter=coloredFormatter
args=(sys.stdout,)

[formatter_coloredFormatter]
class=colorlog.ColoredFormatter
format=# %(log_color)s%(asctime)s - %(name)s - %(levelname)s - %(message)s
datefmt=%Y-%m-%d %H:%M:%S

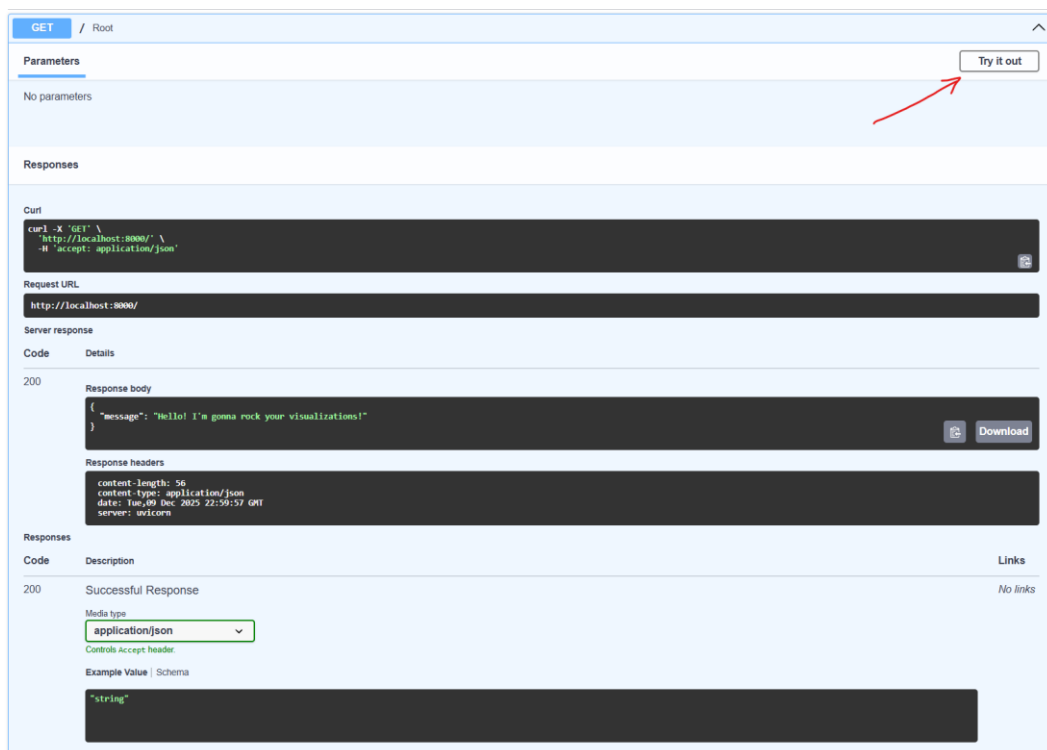
```

```
[colors]
DEBUG=\033[37m
INFO=\033[94m
WARNING=\033[93m
ERROR=\033[91m
CRITICAL=\033[97m\033[41m
```

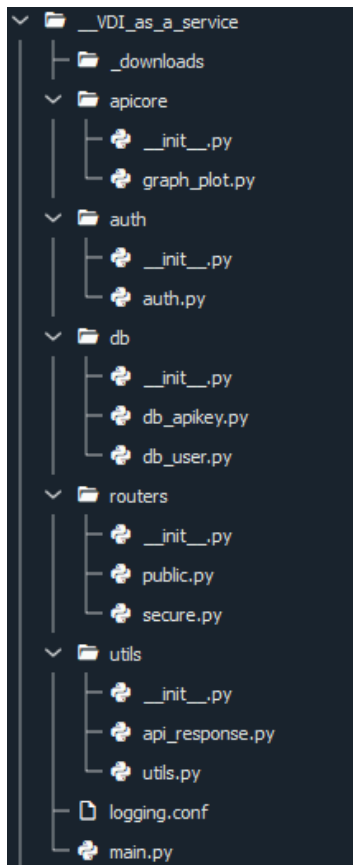
Corra o ficheiro main.py. Num navegador Web da sua preferência, aceda ao endereço <http://localhost:8000/docs>. Resultado esperado:



3. Explore o método *root*.



4. Replique a seguinte estrutura de pastas no projeto FastAPI que iniciou:



5. Siga as instruções que se seguem para disponibilizar um conjunto de serviços públicos.
 - a. Todos os ficheiros `__init__.py` devem conter o seguinte código:

```
import os

ROOT = '__VDI_as_a_service'

parent_dir = os.path.dirname(os.path.realpath(__file__))
while ROOT not in parent_dir.split(os.sep)[-1]:
    parent_dir = os.path.dirname(parent_dir)

if parent_dir not in os.sys.path:
    os.sys.path.append(parent_dir)

print("{} appended.".format(parent_dir))

if os.path.dirname(os.path.dirname(os.path.abspath(__file__))) not in os.sys.path:
    os.sys.path.append(os.path.dirname(os.path.abspath(__file__)))

print("{} appended.".format(os.path.dirname(os.path.abspath(__file__))))
```

Este código fará o carregamento das dependências de todos os módulos automaticamente.

b. O *graph_plot.py* deverá conter o seguinte código:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
from io import BytesIO

def csv_summary(file):
    df = pd.read_csv(file)

    # Build a useful summary
    summary = {
        "columns": list(df.columns),
        "shape": df.shape, # (rows, columns)
        "describe": df.describe(include="all").fillna("").to_dict()
    }

    return summary

def plot_gender_class(file, gender='female', pclass=1):

    df = pd.read_csv(file)

    subset = df[(df['Sex'] == gender) & (df['Pclass'] == pclass)]
    subset = subset[['Sex', 'Age', 'Pclass', 'Survived']]

    # Distribuição de idades por sobrevivência
    plt.figure(figsize=(6, 4))
    plt.title(f"Sobrevivência — {gender} da {pclass}ª Classe")
    sns.histplot(data=subset, x='Age', hue='Survived', kde=True)
    buffer = BytesIO()
    plt.savefig(buffer, format="png", bbox_inches="tight")
    plt.close()
    buffer.seek(0)

    return buffer

def storytelling(file):

    df = pd.read_csv(file)

    # Ajustar estilo
    sns.set(style="whitegrid")

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle("Storytelling do Titanic — 4 Perspetivas", fontsize=16)

    # =====
    # 1. Distribuição das Idades
    # =====
    sns.histplot(df["Age"].dropna(), bins=30, kde=True, ax=axes[0, 0], color="steelblue")
    axes[0, 0].set_title("Distribuição de Idades")
    axes[0, 0].set_xlabel("Idade")
```

```

axes[0, 0].set_ylabel("Frequência")

# =====
# 2. Taxa de sobrevivência por género
# =====
sns.barplot(x="Sex", y="Survived", data=df, ax=axes[0, 1], palette="pastel")
axes[0, 1].set_title("Taxa de Sobrevivência por Género")
axes[0, 1].set_ylabel("Proporção de Sobreviventes")

# =====
# 3. Taxa de sobrevivência por classe
# =====
sns.barplot(x="Pclass", y="Survived", data=df, ax=axes[1, 0], palette="muted")
axes[1, 0].set_title("Sobrevivência por Classe Social")
axes[1, 0].set_xlabel("Classe")
axes[1, 0].set_ylabel("Proporção de Sobreviventes")

# =====
# 4. Relação idade vs sobrevivência
# =====
sns.scatterplot(
    x="Age",
    y="Survived",
    hue="Sex",
    data=df,
    ax=axes[1, 1],
    palette="Set1",
    alpha=0.6
)
axes[1, 1].set_title("Idade vs Sobrevivência")
axes[1, 1].set_ylabel("Sobreviveu (0 = Não, 1 = Sim)")

plt.tight_layout(rect=[0, 0, 1, 0.96])

buffer = BytesIO()
plt.savefig(buffer, format="png", bbox_inches="tight")
plt.close()
buffer.seek(0)

return buffer

```

*nota → o método “storytelling” não será ainda usado nesta fase, mas fica já implementado para poder ser usado nos exercícios seguintes, ligados à proteção dos *endpoints* por *api-key*.

c. O *auth.py*, o *db_apikey.py* e o *db_user.py* poderão ser deixados sem conteúdo para já.

d. O *public.py* deverá ser populado com o seguinte conteúdo.

```

from fastapi import APIRouter

##### UTILS
##### UTILS
##### UTILS
if __name__ == "__main__" and __package__ is None:
    import __init__
##### UTILS

```

```
##### UTILS
##### UTILS
from fastapi import File, UploadFile
from utils.api_response import response_handler
from apicore.graph_plot import *
from fastapi.responses import StreamingResponse

router = APIRouter()

#####
#####
@router.post("/dados_gerais/", tags=["Visualização simples"])
async def dados_gerais(csv: UploadFile = File(...)):
    return response_handler(lambda: csv_summary(csv.file))
#####
#####

#####
#####
@router.post("/sobreviventes_genero_classe/", tags=["Visualização simples"])
def sobreviventes_genero_classe(csv: UploadFile = File(...), genero: str = 'female', classe: int = 1)-
>StreamingResponse:
    return StreamingResponse(plot_gender_class(csv.file, genero, classe),
media_type="image/png")
#####
#####
```

e. O código do ficheiro *api_response.py* deverá ser o seguinte:

```
from fastapi.responses import StreamingResponse

def __response_ok(response):
    if not isinstance(response, StreamingResponse):
        return {'status': 'ok', 'payload': response}
    else:
        return response

def __response_error(response):
    return {'status': 'error', 'payload': response}

def response_handler(fn_exec):
    try:
        response = fn_exec()
        return __response_ok(response)
    except Exception as e:
        return __response_error(str(e))
    #return __response_error(".".join(traceback.format_exception(*sys.exc_info())))
```

Este modulo serve para formatar algumas das saídas produzidas pelos *endpoints* (a implementar mais à frente), que passam a conseguir indicar mensagens de sucesso/conteúdo ou erro/motivo.

- f. O *utils.py*, que opera como um “canivete-suíço” de funções multipropósito, deve conter o seguinte código (embora algumas das funções não sejam efetivamente usadas):

```
import pandas as pd
import datetime
import time
import inspect
import glob, os
import uuid
import socket
import pyzipper
import jellyfish
import hashlib
import json
import re
#####ERROR SECTION

PORT = 8000

def error_101():
    return {"result": "ERROR", "code": inspect.currentframe().f_code.co_name.upper(),
"message": ""}

def check_error(query_data):
    return type(query_data) is dict and "result" in query_data and query_data["result"] ==
"ERROR"

#####SUCCESS SECTION

def success(data):
    return {"result": "OK", "message": data}

#####

ROOT = '__VDI_as_a_service'

def root():
    global ROOT
    parent_dir = os.path.dirname(os.path.realpath(__file__))
    while ROOT not in parent_dir.split(os.sep)[-1]:
        parent_dir = os.path.dirname(parent_dir)
    return parent_dir

def bootstrap():
    if root() not in os.sys.path:
        os.sys.path.append(root())

    print("{} appended.".format(root()))

def download_folder():
    return os.path.join(root(), "_downloads")
```



```

def nest_level(obj):

    # Not a list? So the nest level will always be 0:
    if type(obj) != list:
        return 0

    # Now we're dealing only with list objects:

    max_level = 0
    for item in obj:
        # Getting recursively the level for each item in the list,
        # then updating the max found level:
        max_level = max(max_level, nest_level(item))

    # Adding 1, because 'obj' is a list (here is the recursion magic):
    return max_level + 1

def text_similarity_jaro_winkler(str1, str2):
    return jellyfish.jaro_winkler_similarity(str1, str2)

def list_files_with_extension(folder_path, extension):
    return glob.glob(os.path.join(folder_path, f"*.{extension}"))

def split_from_nth_occurrence(s, char, n):
    char_escaped = re.escape(char)
    matches = list(re.finditer(char_escaped, s))

    if n == 0 or abs(n) > len(matches):
        return (s, "") # Se n for inválido

    # Se n > 0: contar do início
    if n > 0:
        split_index = matches[n - 1].start()
    # Se n < 0: contar do fim
    else:
        split_index = matches[n].start()

    return s[:split_index], s[split_index + 1:]

def convert_quick_and_dirty_int(row, headers, idxmin, idxmax):
    data = list(row[headers[idxmin:idxmax]])
    for i in range(0, len(data)):
        try:
            data[i] = int(round(float(data[i].replace(',', '.'))))
        except:
            pass
    return data

def convert_quick_and_dirty_float(row, headers, idxmin, idxmax):
    data = list(row[headers[idxmin:idxmax]])
    for i in range(0, len(data)):
        try:
            data[i] = (float(data[i].replace(',', '.')))
        except:
            pass
    return data

```

```

def read_CSV(path):
    # Read the csv file
    data = pd.read_csv(path, sep=";", header=0)
    headers = list(data.columns)
    return data, headers

# function to get unique values
def uniquify(list1):

    # initialize a null list
    unique_list = []

    # traverse for all elements
    for x in list1:
        # check if exists in unique_list or not
        if x not in unique_list:
            unique_list.append(x)

    return unique_list

def gen_code():
    return str(uuid.uuid4())

def zip_files(zip_filename, files_to_zip):
    """
    Parameters:
    - zip_filename: The name of the zip file to create.
    - files_to_zip: List of file paths to include in the zip archive.
    - password: The password for the zip file.
    """
    with pyzipper.AESZipFile(zip_filename, 'w', compression=pyzipper.ZIP_DEFLATED) as zipf:
        for file in files_to_zip:
            zipf.write(file, arcname=file.split(os.sep)[-1])

def get_localIP_port():
    return "{}:{}".format(socket.gethostbyname(socket.gethostname()), PORT)

def hash_string(string):
    h = hashlib.new('sha256') #sha256 can be replaced with different algorithms
    h.update(string.encode()) #give a encoded string. Makes the String to the Hash
    return h.hexdigest() #Prints the Hash

def generator_extern_ref():
    h = hashlib.new('sha256') #sha256 can be replaced with different algorithms
    h.update(generator_unique_ID().encode()) #give a encoded string. Makes the String to the Hash
    return h.hexdigest() #Prints the Hash

def generator_apikey():
    return str(uuid.uuid4())

def generator_unique_ID():
    return uuid.uuid4().hex

```

```

def timestamp():
    return time.time()

def date_time():
    x = datetime.datetime.now()
    return "{}-{}-{}".format(x.day, x.month, x.year), "{}:{}".format(x.hour, x.minute, x.second)

async def read_file(file):
    if file.content_type != "application/json":
        raise Exception("Invalid file (JSON expected)")
    contents = await file.read()
    try:
        return json.loads(contents)
    except json.JSONDecodeError:
        raise Exception("Invalid JSON structure")

```

g. Finalmente, o main.py deve ser ajustado para conter o seguinte conteúdo:

```

import uvicorn
from fastapi import FastAPI
import logging
from fastapi.middleware.cors import CORSMiddleware
from routers import public
from utils import PORT

tags_metadata = [
    {
        "name": "Visualização simples",
        "description": "Dados/gráfico simples.",
    },
    {
        "name": "Storytelling",
        "description": "Uma história com dados.",
    }
]

app = FastAPI(debug=True, openapi_tags=tags_metadata)

app.include_router(
    public.router,
    prefix="/api/v3/public"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allows all origins
    allow_credentials=True,
    allow_methods=["*"], # Allows all methods
    allow_headers=["*"], # Allows all headers

```

```

)

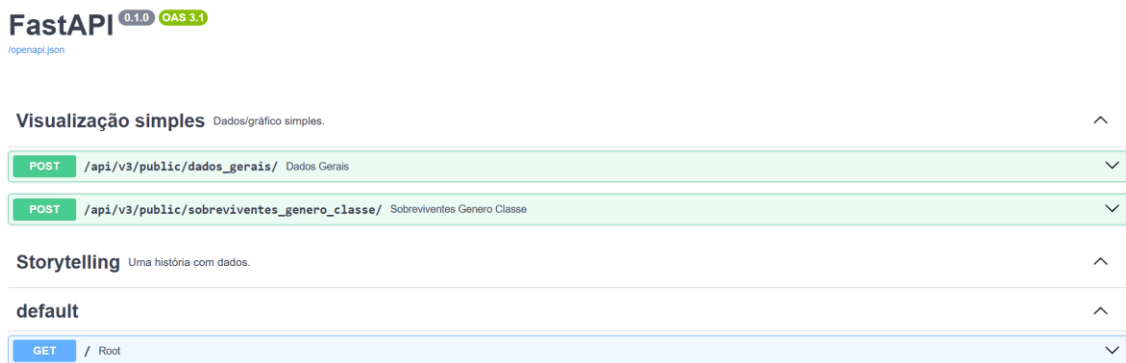
@app.get("/")
async def root():
    return {"message": "Hello! I'm gonna rock your visualizations!"}

if __name__ == "__main__":

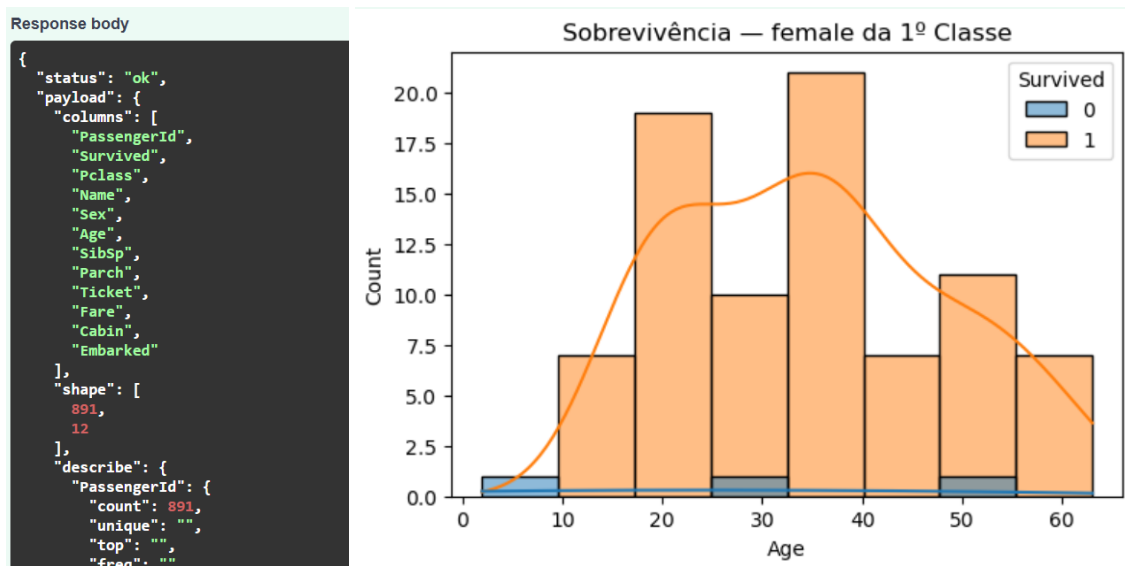
    logging.config.fileConfig('logging.conf', disable_existing_loggers=False)
    logger = logging.getLogger("sampleLogger")
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=PORT,
        log_level=logging.INFO,
        reload=True
    )

```

Agora, ao aceder a `localhost:8000/docs`, deve conseguir ver o seguinte conjunto de serviços públicos:



Recorra ao ficheiro `titanic.csv` que se encontra na BB (na mesma pasta em que encontrou este tutorial), teste os *endpoints* e constate as seguinte saídas:



6. Vamos agora implementar um serviço privado, que possibilite proteger o acesso a um *endpoint* com base numa “pseudo” api-key.
 - a. No ficheiro *auth.py*, introduza o seguinte código:

```
from fastapi import Security, HTTPException, status
from fastapi.security import APIKeyHeader

#####
###
# Allow script to run directly by adjusting sys.path if necessary
if __name__ == "__main__" and __package__ is None:
    import __init__
#####
###

from db.db_apikey import check_api_key, get_user_from_api_key

api_key_header = APIKeyHeader(name="X-API-Key")

def get_user(api_key_header: str = Security(api_key_header)):
    if check_api_key(api_key_header):
        user = get_user_from_api_key(api_key_header)
        return user
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Missing or invalid API key"
    )
```

Este ficheiro será o responsável por validar a autenticação de uma *api-key* que seja passada à autenticação da camada de serviços “secure”.

- b. Agora, coloque o seguinte código no ficheiro *db_apikey.py*:

```
##### UTILS
##### UTILS
##### UTILS
import os, sys
if __name__ == "__main__" and __package__ is None:
    import __init__
from utils.utils import *
##### UTILS
##### UTILS
##### UTILS

from .db_user import get_user_external_ref

from dataclasses import dataclass

@dataclass
class ApiKey:
    apikey: str = None
    akeref: str = None

def check_api_key(api_key: str):
    return api_key == '1111'

def get_user_from_api_key(api_key: str):
    if api_key == '1111':
        return get_user_external_ref('0000')
    return None

if __name__ == "__main__":
    print(get_user_from_api_key('1111'))
```

c. Introduza, também, este código no *db_user.py*:

```
##### UTILS
##### UTILS
##### UTILS
import os, sys
if __name__ == "__main__" and __package__ is None:
    import __init__
from utils.utils import *
##### UTILS
##### UTILS
##### UTILS

from dataclasses import dataclass

@dataclass
class User:
    username: str = None
    timestamp: float = None
    date: str = None
    time: str = None
    validation_timestamp: float = None
    extref: str = None
```

```
def get_user_external_ref(er):
    user = User()
    user.username = "Default"
    if er == "0000":
        user.extref = er
        return user
    return None

if __name__ == "__main__":
    print(get_user_external_ref("0000"))
```

Verifique que o *db_apikey.py* e o *db_user.py* são interdependentes, conciliando o conjunto *referência externa* e *api key*. Foram usados os valores *hard-coded* “0000” e “1111” respetivamente para mitigar a necessidade de ligação a uma base de dados.

d. Agora, no ficheiro *secure.py*, coloque o seguinte conteúdo:

```
from fastapi import APIRouter

##### UTILS
##### UTILS
##### UTILS
if __name__ == "__main__" and __package__ is None:
    import __init__
##### UTILS
##### UTILS
##### UTILS
from fastapi import File, UploadFile
from apicore.graph_plot import *
from fastapi.responses import StreamingResponse

router = APIRouter()

#####
@router.post("/graficos_storytelling/", tags=["Storytelling"])
async def graficos_storytelling(csv: UploadFile = File(...)) -> StreamingResponse:
    return StreamingResponse(storytelling(csv.file), media_type="image/png")

#####
```

e. Finalmente, ajuste o *main.py* para incorporar os dois níveis de roteamento (público e seguro):

```
import uvicorn
from fastapi import FastAPI, Depends
import logging
from fastapi.middleware.cors import CORSMiddleware

from routers import public, secure
```

```

from auth.auth import get_user
from utils.utils import PORT

tags_metadata = [
    {
        "name": "Visualização simples",
        "description": "Dados/gráfico simples.",
    },
    {
        "name": "Storytelling",
        "description": "Uma história com dados.",
    }
]

app = FastAPI(debug=True, openapi_tags=tags_metadata)

app.include_router(
    public.router,
    prefix="/api/v3/public"
)

app.include_router(
    secure.router,
    prefix="/api/v3/secure",
    dependencies=[Depends(get_user)]
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allows all origins
    allow_credentials=True,
    allow_methods=["*"], # Allows all methods
    allow_headers=["*"], # Allows all headers
)

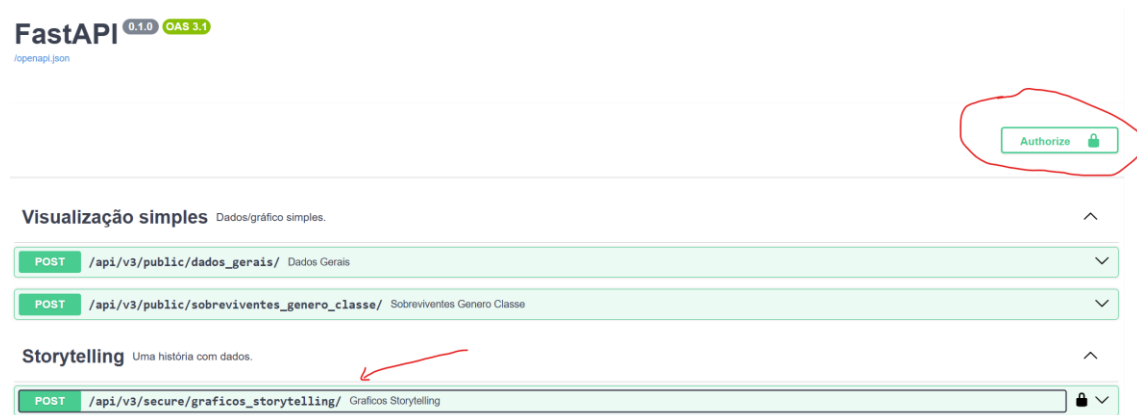
@app.get("/")
async def root():
    return {"message": "Hello! I'm gonna rock your visualizations!"}

if __name__ == "__main__":

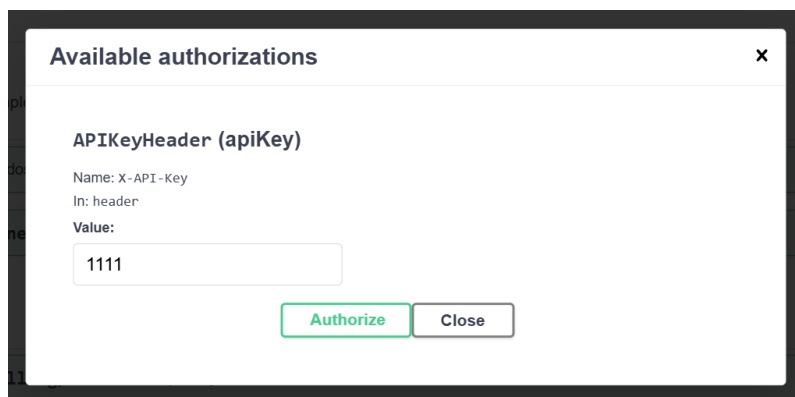
    logging.config.fileConfig('logging.conf', disable_existing_loggers=False)
    logger = logging.getLogger("sampleLogger")
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=PORT,
        log_level=logging.INFO,
        reload=True
    )

```


Verifique o surgimento de um novo botão, no acesso a localhost:8000/docs.

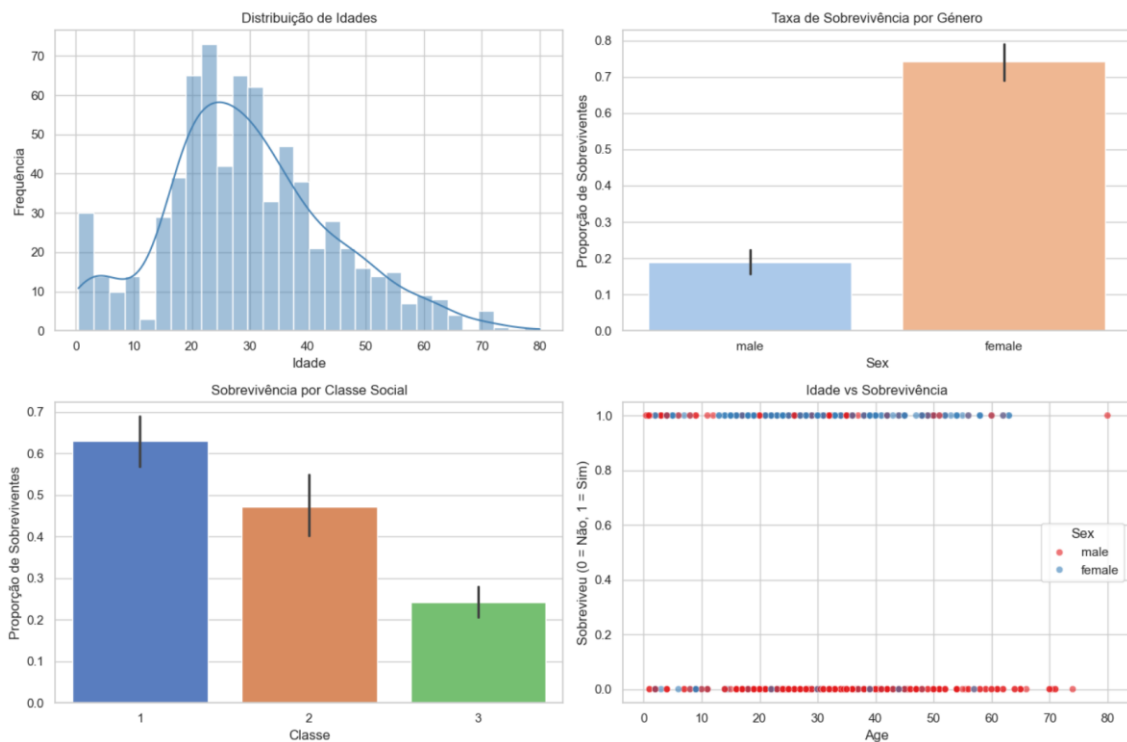


Autentique o acesso com a *api-key* “1111” (outra combinação não irá funcionar).



Experimente o serviço protegido “graficos_storytelling” que surge logo abaixo da secção “storytelling”, alimentando o *endpoint* com o ficheiro “titanic.csv”. O seguinte resultado deve ser produzido.

Storytelling do Titanic — 4 Perspetivas



7. Como exercício complementar ao tutorial apresentado, devem agora “conversar” com uma IA (como o ChatGPT ou o Copilot) para vos ajudar a escolher ferramentas de *frontend* adequadas – por exemplo, Bootstrap para a estrutura e estilo da página, CSS adicional para melhorar a apresentação e bibliotecas JavaScript como jQuery ou Fetch API para consumir os *endpoints* que criaram. O objetivo é pedir à IA sugestões e exemplos de código que permitam receber as visualizações devolvidas pelo vosso *backend* (FastAPI) e apresentá-las corretamente em elementos HTML específicos, criando assim uma pequena interface web funcional para os vossos gráficos e resultados.